Multi-line AI-assisted Code Authoring

Omer Dunay
omerdu@meta.com
Meta Platforms, Inc.
Menlo Park, CA, USA

Daniel Cheng
danielcheng@meta.com
Meta Platforms, Inc.
Bellevue, WA, USA

Adam Tait
adamtait@meta.com
Meta Platforms Inc.
USA

Parth Thakkar
parthdt@meta.com
Meta Platforms, Inc.
Menlo Park, CA, USA

Peter C. Rigby*
pcr@meta.com
Meta Platforms, Inc.
USA

Andy Chiu
achiu@meta.com
Meta Platforms, Inc.
USA

Imad Ahmad
imadahmad@meta.com
Meta Platforms, Inc.
Menlo Park, CA, USA

Arun Ganesan
arunganesan@meta.com
Meta Platforms, Inc.
USA

Chandra Maddila
cmaddila@meta.com
Meta Platforms, Inc.
Bellevue, WA, USA

Vijayaraghavan Murali
vijaymurali@meta.com
Meta Platforms, Inc.
Menlo Park, CA, USA

Ali Tayyebi
alitayyebi@meta.com
Meta Platforms, Inc.
New York, NY, USA

Nachi Nagappan
nnachi@meta.com
Meta Platforms, Inc.
USA

arXiv:2402.04141v1 [cs.SE] 6 Feb 2024

ABSTRACT

CodeCompose is an AI-assisted code authoring tool powered by large language models (LLMs) that provides inline suggestions to 10's of thousands of developers at Meta. In this paper, we present how we scaled the product from displaying single-line suggestions to multi-line suggestions. This evolution required us to overcome several unique challenges in improving the usability of these suggestions for developers.

First, we discuss how multi-line suggestions can have a "jarring" effect, as the LLM's suggestions constantly move around the developer's existing code, which would otherwise result in decreased productivity and satisfaction.

Second, multi-line suggestions take significantly longer to generate; hence we present several innovative investments we made to reduce the perceived latency for users. These model-hosting optimizations sped up multi-line suggestion latency by 2.5x.

Finally, we conduct experiments on 10's of thousands of engineers to understand how multi-line suggestions impact the user experience and contrast this with single-line suggestions. Our experiments reveal that (i) multi-line suggestions account for 42% of total characters accepted (despite only accounting for 16% for displayed suggestions) (ii) multi-line suggestions almost doubled the percentage of keystrokes saved for users from 9% to 17%. Multi-line CodeCompose has been rolled out to all engineers at Meta, and less than 1% of engineers have opted out of multi-line suggestions.

1 INTRODUCTION

CodeCompose [15] provides inline suggestions as a software engineer types code, but it was originally only designed to predict tokens that would complete the current line. Such single-line suggestions should be quick, highly accurate, and help with the immediate context. In contrast, multi-line suggestions need to be deeply informed and intelligent. These code blocks can help users discover APIs, best practices, and implementation details. In this work, we describe how we added multi-line suggestions to CodeCompose.

CodeCompose suggests code as the user types, impacting the main authoring workflow. Multi-line suggestions can have a "jarring" effect as the displayed suggestions move around code that the user has already written. This mingling of suggested code with human written code has a high cognitive load on the user, as they have to review and rework code that they trust and wrote, with code that they need to verify. We adhered to strict design principles of not being intrusive, and of fitting seamlessly into the existing user experience.

There were three main challenges in developing multi-line suggestions at scale: (1) eliminating the jarring effect (2) providing responsive suggestions with low latency for large blocks of code to be generated (3) rolling out and evaluating the impact of multi-line to 10's of thousands of developers.

Challenge 1. User interface experience - Eliminating the "jarring" effect

Avoiding the "jarring" effect is simple with single-line suggestions: we only show suggestions when the cursor is at the end of a line (with the exception of special characters at the end like brackets). In contrast, multi-line is complex as it can disrupt code that is already written. Creating an algorithm for this solution poses a non-trivial technical problem, including figuring out how to determine the context of the cursor reliably. Moreover responses from the LLM can be not well formatted, and aligning them into the cursor position may cause further distraction for the user.

CodeCompose handles these challenges with a combination of pre-processing and post-processing algorithms that utilize a semantic context understanding of the programming language and cursor position scope. Section 3 discusses this multi-line algorithm based on the semantic scope to minimize noise to the user.

*Rigby is a professor at Concordia University in Montreal, QC, Canada.

Challenge 2. Responsive User Experience

CodeCompose's inline suggestions show up automatically as the user types. Each suggestion aligns with a given state of the active file, and is being invalidated as soon as the user types an additional keystroke or moves their cursor. Multi-line suggestions are long by nature; therefore it may take a few seconds for the LLM to generate them, during which time the user may hit another keystroke and dismiss the response before they have even seen the suggestion.

Since latency is a key factor in determining the "display rate", *i.e.* the number of suggestions that users are actually able to view, we invested in reducing the latency of long multi-line suggestions through both the client extension and the model-hosting service, as outlined in Section 4.

Challenge 3. Production release and effectiveness measurement

The final challenge we faced was in monitoring the effectiveness of the experimental features enumerated in the prior sections. We tracked a host of metrics online to evaluate whether each feature was improving the user experience, and adjusted the product accordingly. In particular, we needed to evaluate whether users found multi-line suggestions more useful compared to only single-line suggestions, due to the higher latency and reduced display rate of multi-line suggestions.

These metrics included acceptance rate, display rate, latency, % keystrokes saved, and # of characters accepted per user. These were broken down by single-line vs multi-line suggestions to determine the net benefit of each feature for users.

Result Summary. In monitoring our online A/B tests, we found that the investment in multi-line suggestions disproportionately increased throughput, as:

- Multi-line suggestions accounted for 42% of total characters accepted day (despite only accounting for at 16% of displayed suggestions)
- Multi-line suggestions drove a significant net increase in % keystrokes saved from 9% to 17%, as shown in Table 1

This paper is structured as follows. In Section 2, we give background on Meta and introduce our experimental methodology. In Sections 3 to 5, we address each challenge. In Section 6, we discuss threats to validity. In Sections 7 and 8, we discuss related work and conclude the paper.

2 BACKGROUND AND METHODOLOGY

2.1 Meta

Meta is a major industrial software organization with a complex code base that covers a wide range of applications ranging from social networking and virtual reality to software engineering infrastructure, such as continuous integration tooling and workplace coordination. Thousands of developers work on billions of lines of code (LOCs) in a monolithic repository that contains source code written in tens of programming languages. At Meta, code authoring is one of the prominent activities in the Software Development Life Cycle (SDLC). A significant amount of contextual knowledge on the internal software development processes, and libraries, is contained within in the source code or is confined to a small set of developers. For example, a question like *"How can I query a table from Hive in Hack?"* can be answered using that knowledge. The state of the source code and the developers associated with the source code are in a constant state of evolution - internal frameworks and tools get added and deprecated, while developers move across teams and change roles. At Meta's scale, keeping up with the knowledge required to accomplish coding tasks is a major challenge. Additionally, the dynamic environment at a large software company like Meta poses several challenges concerning knowledge discovery and improving developer productivity.

2.2 CodeCompose at Meta

At Meta, we built an AI-assisted code authoring system named CodeCompose to explore the application of LLM technology for code authoring [15]. Our team conducted R&D on the underlying LLM architecture and converged on using InCoder-6.7B [13] as the base model for the initial version of CodeCompose. Recently, with the release of CodeLlama [19], we switched to using a fine-tuned version of CodeLlama-7B as the foundation model.

CodeCompose has several desired characteristics for powering an inline suggestion experience: (i) multi-linguality, due to the CodeLlama's training on a multitude of programming languages, (ii) customized for Meta, as a result of our internal fine-tuning of CodeLlama on organization-specific data, (iii) natural language proficiency, which gives it the capability to generate and understand inline code comments, and (iv) bi-directionality, due to its fill-in-the-middle (FIM) training that allows it look at both the code before and after the cursor when generating a suggestion.

2.3 Measures for Evaluating CodeCompose

To evaluate CodeCompose we used the following measures:

- # of suggestions displayed per user per day
- E2E latency of generating and displaying suggestions
- Acceptance rate (# accepted / # displayed for suggestions shown to user for > 750ms)
- # chars accepted per user per day
- % keystrokes saved saved (# chars accepted / # of chars typed by user)

We think of these metrics as a funnel, with suggestions displayed at the top, leading to increased throughput in suggestions accepted, and increasing the total % of keystrokes saved. Latency serves as a guardrail metric, but also feeds into this funnel since lower latency increases the # of suggestions displayed.

At Meta, changes are rolled out in randomized double blind trials, *i.e.* A/B tests [20]. We describe the setup for each of our rollouts to evaluate the effectiveness of each new multi-line feature against the holistic user experience in Section 5.

3 ADDRESSING CHALLENGE 1

User interface experience - Eliminating the "jarring" effect

In this section, we explore the unique challenges and considerations associated with deploying a coding assistant capable of multi-line suggestions within a large-scale industrial organization like Meta. These insights are derived from feedback received from hundreds of users that were using CodeCompose as early adopters. Contrary to initial intuition, the multi-line use case presents a higher level of complexity compared to the single-line use case,

```
# single line jarring example:
def quicksort(n):quicksort(arr):
```

Figure 1: Single-line "jarring" effect example: The user cursor positioned between "def" keyword and the "quicksort" function, inline suggestion appears and moves the existing user code to the right.

from both a product perspective and technical implementation standpoint. The primary objectives for each use case are as follows:

- **Single-line**: The aim is to provide quick and highly accurate assistance, facilitating task completion and reducing the burden of keystrokes for straightforward and repetitive tasks.
- **Multi-line**: The goal is to offer deeply informed, intelligent assistance that aids users in discovering APIs, best practices, and implementation details.

We provide examples of the single-line completion in Figure 1, the difficulties in multi-line completion in Figure 2, and our multi-line strategy in Figure 3. The captions for the figures are extensive to allow the reader to walkthrough the examples.

3.1 Definition of the "jarring effect"

CodeCompose suggests code as the user types. This means suggestions appear frequently while the user is engaged in their primary authoring workflow. One of our strictest design principles is that suggestions should not be intrusive and should seamlessly integrate into the user flow. Based on user feedback, we identified suggestions that significantly disrupt the user, resulting in a jarring experience. The two primary instances of this are (1) suggestions that shift existing code that the user is currently reading or typing, either upwards, downwards, or to the right as shown in Figure 1 (2) suggestions that do not align with the structure of the code, such as poorly formatted suggestions that overlap the existing scope of the cursor as shown in Figure 2.

3.2 Approach to address the "Jarring Effect"

Addressing the "jarring effect" for *single-line* suggestions is relatively uncomplicated. To prevent the displacement of existing code, we simply refrain from displaying suggestions if there is any code to the right of the cursor, with certain exceptions such as },) and]. Since no characters exist on the right-hand side of the cursor when a suggestion appears, no user code can be shifted to the right. Furthermore, as we only suggest until the end of the line, we eliminate the risk of introducing unformatted code that overlaps the existing cursor scope. However, for multi-line suggestions, the "jarring effect" is more likely to occur, as existing code below the cursor is constantly being pushed up and down.

To circumvent the jarring effect for multi-line suggestions, we adhere to the following rules:

- Multi-line suggestions are triggered only when the cursor is positioned at the end of the scope that owns the cursor.
- Multi-line suggestions should be shown until the end of the current scope.

```
# quicksort implementation
def  quicksort(n):

    if len(n) <= 1:
        return n

    pivot = n[0]
    left = [x for x in n[1:] if x < pivot]
    right = [x for x in n[1:] if x >= pivot]

    return quicksort(left) + [pivot] + quicksort(right)

    test1 = 1 # (this line should not jump)
```

Figure 2: Example showing multi-line "jarring" effect: the user cursor was between a function name and the next line containing the statement "test1 = 1". When the suggestion occurs, the existing line is pushed down disrupting the developer's flow and forcing them review the suggested "quicksort" function while also determining the correct location of their existing code.

When users write code, their flow and mindset is in the most inner scope. Therefore, satisfying rule (1) ensures that when we suggest multi-line to the user, the lines below that are pushed down are in the outer encapsulating scope, thereby not disrupting the user flow and not causing the jarring effect. Furthermore, satisfying rule (2) ensures the suggestion structure is completing until the end of the current scope with no overlap.

We specifically trigger multi-line in the following cases:

(1) The cursor is positioned at the end of the most inner scope that contains it.
(2) The cursor is at the end of a line that defines a new symbol that creates a new scope.
(3) The cursor is at the end of a notebook cell (for Bento [1] notebook use case).
(4) The user explicitly requested using a shortcut key.

3.3 Technical Implementation of the Strategy

The requirement to trigger multi-line only in certain cases, based on the cursor position, necessitated the integration of semantic context understanding into the CodeCompose system. The existing CodeCompose system employs a typical client-server architecture (shown in Figure 4) in which the server is an inference tier that runs the model and a client is an editor that surfaces code suggestions. We encode the bulk of the client-side logic in a Language Server Protocol (LSP) conformant language server that is reused across multiple editor integrations. To mediate requests between the client and server, we implemented a language server in Rust that we reuse across our various editor integration. While most language servers are designed to support a wide array of traditional IDE functionality (autocomplete, jump-to-definition, etc.), the CodeCompose language server supports only one meaningful request type: "textDocument /inlineCompletions".

```
function foo() {
  ▌                    ✓ Multi-line will be shown
}
```

(a) Multi-line suggested since the cursor is positioned at the end of an empty function which is the inner scope.

```
function foo() {
    const a = 1;
    ▌                  ✓ Multi-line will be shown
}
```

(b) Multi-line suggested since the cursor is positioned at the end of an non-empty function which is the inner scope.

```
function foo() {
    const a = 1;
    if (a = 1){
        // some comment
        ▌              ✓ Multi-line will be shown

    }
    const b = 2;
}
```

(c) Multi-line suggested since the cursor is at the end of "if statement" which is the inner scope.

```
function foo▌          ✓ Multi-line will be shown
```

(d) Multi-line suggested since the cursor is at the end of a newly defined function.

```
function foo() {
    const a = 1;
    if (a = ▌          ✓ Multi-line will be shown

    const b = 2;
}
```

(e) Multi-line suggested since the cursor is at the end of a newly defined "if statement".

```
function foo() {
    ▌                  ⚠ Single-line will be shown
    const a = 1;
}
```

(f) Multi-line is not suggested since the inner scope if function and the cursor is not at the end of it.

```
function foo() {
    const a▌= 1;       ✗ No suggestions shown
}
```

(g) Both multi-line and single-line not suggested since there are characters after the cursor in the same line.

Figure 3: Examples showing pre-processing stage: Deciding based on the cursor position which type of suggestion should be displayed.

Avoiding the jarring effect for single-line was trivial from a technical point of view as it required a simple conditional statement. For multi-line on the other hand, the strategy described above is much more complex and requires detailed semantic context understanding of the programming language.

Request workflow - Pre-processing: A completion request is sent from the client to the language server, the CodeCompose language server parses the current state of the file and understands the cursor position. If the cursor's position aligns with one of the cases outlined above, this request is marked as a multi-line request. The request is then validated in the local cache; if it is a cache miss, it is sent to the CodeCompose model flagged as a multi-line request (examples shown in Figure 3).

Request workflow - Post-processing: The model LLM generates suggestions and sends back the response to the language server. The model response does not have a strict guarantee for how well it will be structured; it may contain overlap with code in the current scope. Therefore, in the language server, the response goes through an additional step of post-processing that truncates the response in case it overlaps the current scope, as shown in Figure 5.

Finally, we built an extensive unit test suite (473 tests) to validate that there was no end-to-end jarring effect or incorrect truncation with our implementation.

> In summary: (1) Multi-line suggestions are triggered only when the cursor is at the end of scope (2) Suggestions are shown until the end of the current block (3) After the suggestion is accepted, the cursor needs to be moved to the end of the suggested block.

4 ADDRESSING CHALLENGE 2

Responsive User Experience

CodeCompose's inline suggestions show up as the user types, each suggestion aligns with the given state of the file and invalidated (dismissed in the editor's User Interface) as soon as the user types an additional keystroke or moves their cursor. This implies that latency is a key factor in determining the "display rate" - the percentage of suggestions shown to users.

For single-line we limit the generation of a suggestion to stop at either newline or 25 max tokens. The average latency is low, therefore display rate is not a major concern. Multi-line generations are much longer (p50 325 chars, p90 450 chars), and max tokens is set to 120. This can take over 2 seconds for large suggestions; therefore, reducing latency for multi-line is significantly more likely to increase the display rate. In this section we describe multiple approaches and projects that were built focused on reducing the latency of generating long multi-line generations and increasing its display rate. The improvements were done across all main components of the CodeCompose System - the client-side editor extension, the language server and the service hosting the model.

4.1 Improvements in the editor client extension (i.e. VSCode / Bento [1] Notebooks)

During initial testing, CodeCompose's multi-line suggestions were generated automatically as the user typed. However, from the user's

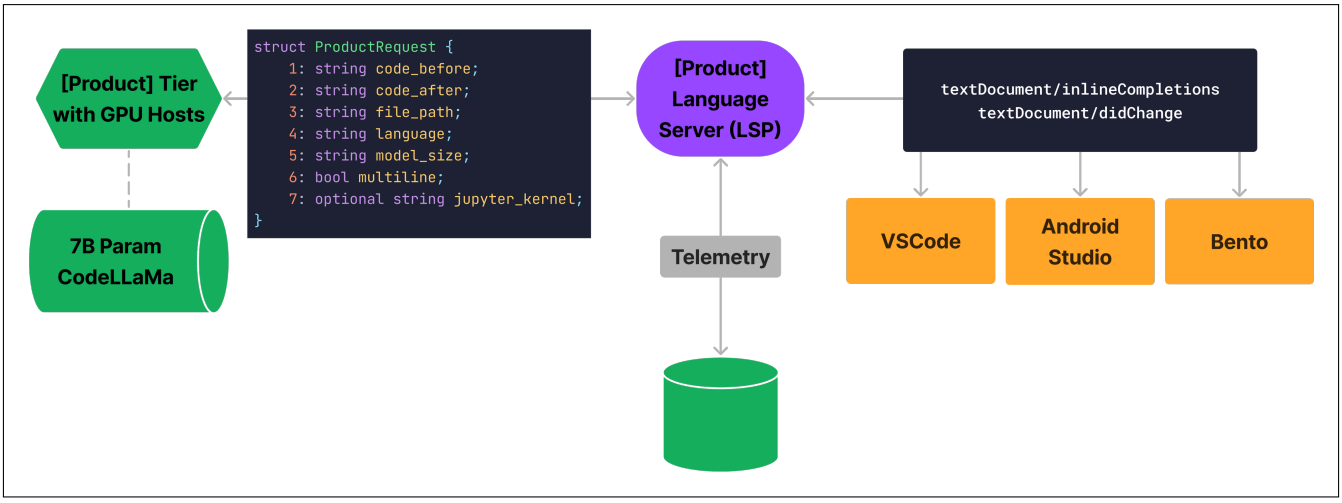

Figure 4: System architecture of CodeCompose: Client editor that surface the suggestions, a language server to mediate requests with CodeCompose model service host. In the request "multi-line" flag is passed to the model service.

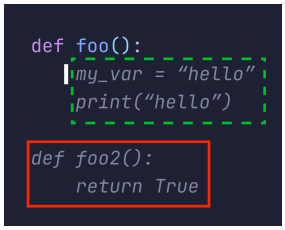

Figure 5: Example showing the post-processing stage: The cursor is in the scope of the "foo" function. Although, the model returns a multi-line suggestion of both the "foo" and "foo2" functions, postprocessing will remove the code in the red box, and will only display suggestions for the in-scope "foo" function to the user.

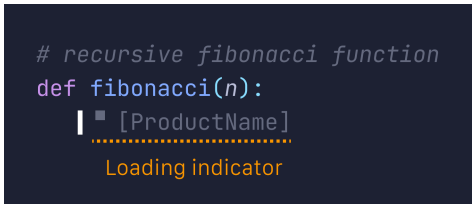

Figure 6: Inline indicator (marked with orange underline) with a spinner shown to the left of the cursor alerting the user CodeCompose may show soon an AI suggestion

.

perspective, there was not a deterministic way to know whether a CodeCompose suggestion was still being processed and would appear momentarily, or whether CodeCompose had decided not to display a suggestion. This unpredictability caused frustration to the user and made the system appear unreliable.

To solve this, we introduced an inline spinner indicator, as seen in Figure 6, which pops up next to the user cursor as long as there is an active completion that still needs to be responded to. As soon as the language server's pre-processing (described in 3) determines that the current request should be multi-line, it sends a "CodeCompose/fetchingMultiline" notification to the editor client, which then renders the indicator. The editor dismisses the indicator once the request is responded to or canceled.

The user's mental model is rooted in knowing whether a suggestion is expected to appear or not. The inline indicator lets users know if a multi-line suggestion is being processed, and therefore allows them to make a decision on whether to type the next character – or to wait a little longer to see the upcoming suggestion. This feedback improves the overall display rate by nudging users to wait a little longer, especially for longer multi-line suggestions. This results in a transparent, predictable experience for users that helps build their trust in the CodeCompose system.

> Because multi-line suggestions take significantly longer to generate, we added a spinning UI indicator so that users were aware that a multi-line suggestion was being generated. This reduced the perceived latency and increased usage of multi-line suggestions over single-line suggestions.

4.2 Optimizations to the model-hosting service

We implemented numerous improvements to the model-hosting service with the primary goal of reducing latency. Our hypothesis was that latency reduction would increase all of our quantitative outcome metrics, particularly for multiline suggestions where generation time was significantly longer. Suggestions being available sooner would mean that more suggestions were shown to users, before they would have taken another action (e.g. typing the next character) that would cause the suggestion not be shown.

From mid-June through September, we ran a series of A/B experiments [20] on CodeCompose, which is rolled out to 10's of thousands of Meta's engineers, that incrementally improve latency: We initially rolled out to 10% of the population as a smoke test, before comparing metrics at 50% rollout for a one week duration to validate the latency impact across all our outcome metrics. Below, we highlight each of the latency experiments we ran, then summarize the net metric gains in Section 5.

Flash Attention. When we implemented Flash Attention [12] on our 6.7B InCoder-based FIM-trained model on multi-line suggestions, we saw a median 2.53x reduction in first token latency and median 1.15x reduction in latency to the final token. Flash Attention is an improvement primarily in the model's attention stack, so a greater improvement in first token latency was expected. In our experiment, we found a 13% increase in the number of suggestions displayed, an 8% increase in acceptance rate, a 3% increase in the average length of suggestions accepted. Overall, we saw a 74% reduction in median latency because Flash Attention improved single-line suggestion latency even more than multi-line and single-line suggestions are the majority of suggestions that our models generate.

CUDA graphs and fused kernels. Using CUDA graphs and fused kernels[8] together on our 6.7B InCoder-based FIM-trained model, we observed that a reduction in our median generation latency by 29%, and at the 90th percentile a reduction in latency of 40%. The latency reductions led to increases in our absolute number of displayed suggestions by 26% and total number of suggestions accepted by 17%. However, we also observed the acceptance rate decline of 1%. From this experiment, we learned that acceptance rate is not strictly correlated with latency. We hypothesize that the improvement in latency during this experiment resulted in suggestions being shown in environments of increasingly low acceptance rates. For example, if the user rejected a suggestion then typed only one more character, a newly generated suggestion was unlikely to be accepted even if displayed immediately with low latency.

Queue Priority. We studied the effect of the longer multi-line suggestions on our GPU capacity utilization. We discovered that longer generations have an outsized effect on capacity load. Additionally, requests for longer multi-line suggestions were receiving time-out responses at nearly 2x the rate of single-line requests. We suspect the reason for the large difference between time-out rates is created by the triggering strategies we employ (as described in Section 3). To alleviate the queue contention, we implemented QoS to advantage our longer multi-line requests. We increased the queue gestation time relative to shorter single-line requests to boost the multi-line success rate. This effort succeeded in reducing the magnitude of multi-line time-out responses. We succeeded in increasing the number of displayed multi-line suggestions and observed a 1% absolute increase in keystrokes saved.

Streaming and early cancellation. Building on our work to truncate generated suggestions at the end of scope (see Section 3.3), our analysis discovered that 54% of all characters generated were being truncated before being shown. Further, 47% of all suggestions generated by the model were never displayed. To reduce this wasted effort by our GPU capacity, we introduced a streaming response mechanism that allowed early cancellation when the client closes the stream. With streaming, the client is able to communicate back to the model service that either the generated suggestion is no longer needed at all or that the model has generated characters that will be truncated. We observed a 45% improvement to overall model service request round trip latency as a result of implementing streaming with cancellation.

Batching. We used 2-way tensor parallelism to serve responses 1.5x faster. Since this uses two GPUs to serve a request, but does not improve the latency by a factor of two, it effectively requires more capacity to handle the same workload. However, we found that this can be more than compensated by using continuous batching, giving us significantly faster responses while also gaining improved effective capacity due to the efficacy of batching multiple requests.

Improving latency has a compounding effect. Reducing latency distributions increases the effective capacity of our machines to serve more requests. For instance, the above optimizations we can serve the same request with a 2.5x reduction in the number of GPUs. The main reason is that there is less wasted computation on cancelled requests and request can be batched. Reducing capacity utilization further reduces queuing times and the distribution of round trip latency.

By combining streaming alongside tensor parallelism and continuous batching, we sped up the median singleline suggestion from 440ms to 280ms (1.5x faster/35% latency reduction), and the median multi-line suggestion from 2000ms to 750ms (2.5x faster/60% latency reduction). This in turn led to a 16% relative improvement in characters accepted by users. This confirms that latency has a significant impact on the effectiveness of code completion tools.

> We invested in five techniques to optimize the model-hosting service, which reduced median latency of multi-line suggestions down to 750ms. This in turn increased the # of suggestions shown to users and the overall keystrokes saved by users, as validated through A/B experiments in Section 5. Having a responsive UX was crucial to the adoption of multi-line suggestions.

5 ADDRESSING CHALLENGE 3

Production release and effectiveness measurement

The final challenge faced was in monitoring the effectiveness of the experimental features enumerated in the prior sections. We tracked a host of metrics in real-time to evaluate whether each feature was improving the user experience, and adjusted the features accordingly. These metrics included acceptance rate, display rate, latency, % keystrokes saved, and # of characters accepted per user. The metrics were broken down by single-line vs multi-line suggestions to determine the net benefit of each feature for users. We ran A/B experiments with a 50/50 split to control and test for each feature release.

5.1 Experiments for Release of Multi-line Suggestions

First, when rolling out each of the multi-line trigger points (as explained in Figure 3), we closely monitored both acceptance rate and throughput for any negative impact on the user experience:

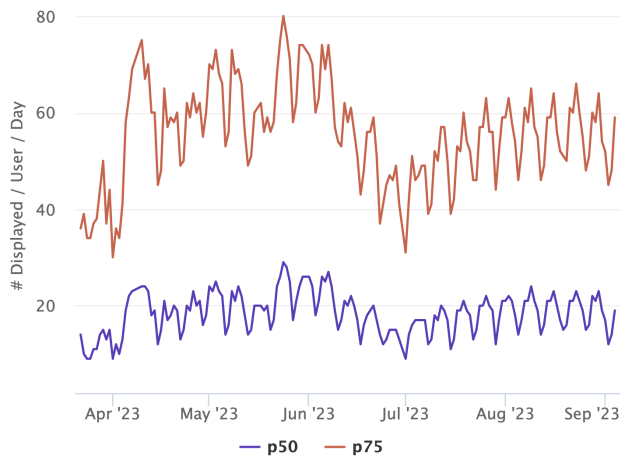

Figure 7: The # of displayed suggestions per user per day, at p50 and p75. This display metric dropped in mid-June as higher latency multi-line suggestions rolled out, but recovered by end of August due to our investments in perceived latency.

(1) Multi-line suggestions require more mental effort for users, as they must review multiple lines rather than reading a single-line suggestion
(2) Multi-line suggestions cause a substitution effect – some of the trigger points (e.g. upon hitting newline in a new function block) overlap with single-line. Due to the longer latency for generating multi-line suggestions, users may end up typing past these trigger points, causing a missed display opportunity for CodeCompose to save the users on typed keystrokes (whereas a single-line suggestion would have displayed, given its lower latency)
(3) Users may sometimes prefer seeing consecutive single-line suggestions, instead of one or two longer multi-line suggestions, depending on their workflows

As seen in Figure 7, when initially rolling out multi-line suggestions in mid-June, the number of displayed suggestions per user dropped significantly, due to the longer generation time which led to users typing past the trigger point.

However, due to our investments in (1) reducing median multi-line latency from 2000ms to 750ms (2) showing the inline indicator (both of which were described in Section 4), this display metric gradually restored close to original levels by end of August. These latency investments decreased the gap in perceived latency between single-line and multi-line suggestions, which in turn contributed to the continued increase in throughput as users found multi-line suggestions more and more valuable with the increased responsiveness of the suggestion UI.

Through our production monitoring, we found that acceptance rate also remained similar at 29% between the two suggestion types, reinforcing that despite the longer suggestions (which are harder to predict accurately), multi-line suggestions were still favored at similar rates as single-line. Hence overall throughput increases since more lines of codes were accepted at a given display opportunity.

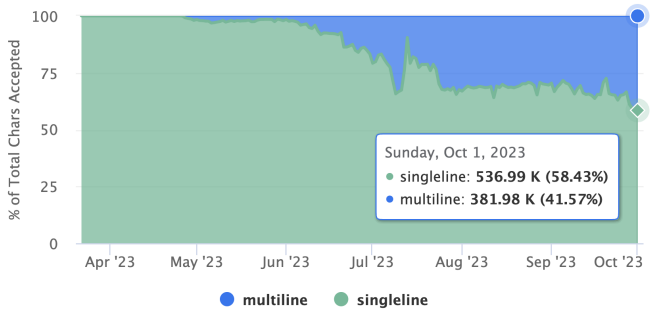

Figure 8: Total characters accepted, broken down by suggestion type. Multi-line suggestions trigger with higher latency and lower frequency (16% of total volume of displayed suggestions). Yet multi-line suggestions account for 42% of total characters accepted for users.

	Baseline	UX Product Release (mid-June)	CodeLlama Model Release (late Aug)	CodeLlama Relative Increase
Single-line	9%	8%	10%	25%
Multi-line	0%	5%	7%	40%
Total	9%	13%	17%	30.70%

Table 1: The % keystrokes saved at each step of the release cycle. (a) The combined effect from the UX Product Release (mid-June to August) was a 5% absolute increase from multi-line, while taking away only 1% absolute from single-line, demonstrating a net improvement to the user of 4% absolute. (b) A new model release (CodeLlama-7B) further showed the value of multi-line as the improvement surface going forward, as the new model increased single-line throughput from 8% to 10% (+25% relative), but a much larger increase of 5% to 7% (+40% relative) for multi-line.

Furthermore, since we triggered multi-line suggestions less frequently to reduce the jarring effect, multi-line suggestions only accounted for roughly 16% of total volume of suggestions displayed. Yet as shown in Figure 8, multi-line suggestions accounted for 42% of total characters accepted by users. These throughput metrics demonstrated the success of our investments in reducing user perceived latency, which in turn increased usage of multi-line suggestions relative to single-line suggestions.

Finally, our online monitoring metrics demonstrate that multi-line suggestions are the correct surface for further improvement to the product over time. As shown in Table 1, when releasing the CodeLlama-7B model, we observed that the impact on multi-line was much greater: +40% relative improvement in multi-line throughput, compared to only +25% relative improvement in single-line throughput. This opens the door to further investments in generating longer and longer suggestions – for example, by increasing the max token length for generation, or by triggering multi-line suggestions at more locations.

5.2 User Feedback and Opt-out Rate

We monitored qualitative user feedback to triangulate the metric improvements we observed in the prior section. Before CodeCompose handled the "jarring" effect, and before new CodeLlama-7B was deployed, several users complained about the "jarring effect" and quality of multi-line suggestions:

- *"[Negative] Lately, I've been finding Python multi-line suggestions distracting. This DevX feels substantially different than single-line completions: It shifts my code around, causing me to shift my gaze from the line I'm writing to figure out what happened. Sometimes the code is useful, but it's largely been not useful compared to single-line completions (I imagine we see this in suggestion accept rate data). I wonder how we can make this DevX less interruptive, while we work on improving suggestion quality."*

After developing CodeCompose's multi-line algorithm, the feedback was much more positive:

- *"[Positive] I had so many moments of surprise and delight to the point of taking screenshots of suggestions. CodeCompose beginning to automate so much of the boring/painful parts of coding to make it super fun & fast. I haven't felt a magical productivity leap like this since <infra feature change from several years ago>."*
- *"[Positive] I had no idea <language> had multi-line strings; this popped up when in a situation where I was better off using one and was grateful for the discovery"*
- *"[Positive] Regarding CodeCompose: Have I become that predictable as a human being! My intention was to write a Python function that generates a SQL case-when statement based on a dictionary I previously defined. I typed the name of the function and it autogenerated the 5-line return statement. It correctly guessed I wanted to create a SQL case-when statement and wrote the function just like I was thinking in my head!!! I always thought the way an individual writes code is unique to that individual! So have I become that predictable as a human being! What even is real, anymore!"*
- *"[Positive] Dealing with annoying boilerplate, ex. propagating 5 variables through state -> query -> response. CodeCompose makes faster work of these than my usual copy+paste+select+change motion [...] Both writing the generic function & replacing the code was 50%+ accepted suggestions"*
- *"[Positive] CodeCompose is just amazing when writing generic functions like this. Of course, this isn't perfect since it did not get the else part of the function correct but still helped me reduce my keystrokes by 90%."*
- *"[Mixed] I've been noticing several cases where the first lines of the CodeCompose suggestion are quite good, but the suggestion gets a bit too ambitious and goes on several lines beyond that with code that is subjectively not what I want and objectively less related to the intent I expressed with what I had typed so far."*
- *"[Negative] I find the multi-line suggestions are rarely useful, and cause issues if I accidentally accept them."*

User feedback underscores the intricate equilibrium required when launching a feature like multi-line suggestions. This balance is between recall – as manifested in the display rate and the number of lines in each suggestion – and the quality of the suggestion itself. As we enhanced the quality of multi-line suggestions by mitigating any jarring effects and improving the model's quality, we observed a corresponding improvement in user feedback. While we leave a full thematic or ground theory analysis of the qualitative feedback as future work, the initial user feedback has been overwhelmingly positive, as shown in the above testimonials.

We also note that very few engineers disabled the multi-line feature. The overall opt-out rate of CodeCompose stayed constant over time at $< 0.8\%$ of all users. We allowed users to specifically toggle off multi-line suggestions, while keeping single-line suggestions on. The multi-line opt-out rate here is even lower at $< 0.1\%$ of all users, indicating widespread adoption and favorability toward multi-line suggestions.

> During the roll-out of multi-line suggestions, we monitored metrics like acceptance rate, display rate, latency, and throughput to evaluate the net benefit of multi-line suggestions vs. single-line suggestions. These metrics demonstrated that the investment in multi-line suggestions disproportionately increased throughput, as multi-line suggestions accounted for 42% of total characters accepted per user per day (despite only accounting for at 16% of displayed suggestions), and drove the significant net increase in percentage of keystrokes saved nearly doubling from 9% to 17% (as shown in Table 1). Less than 1% of engineers opted out and disabled multi-line suggestions.

6 THREATS TO VALIDITY

6.1 Generalizability

This study is conducted at Meta. Although this is a single company it includes codebases many times larger than those typically studied in open source projects. It also contains code that covers a wide range of software systems from social network to infrastructure projects, such as databases and continuous integration systems. There is also software that runs directly on VR hardware. We are able to release CodeCompose to 10's of thousands of engineers. Our results may not generalize to smaller companies and projects that do not have a large codebase to train their models on.

6.2 Construct and Internal Validity

In Section 2.3, we describe the metrics we track to determine if multi-line CodeCompose is effective. There have been few large scale deployments of AI-assisted, so we used the existing infrastructure at Meta to track: display rate, latency, % keystrokes saved, # of characters accepted per user, and opt-out rate. In our A/B tests across 10's of thousands of engineers, the results were positive for each metric and convincing enough to allow us to roll out multi-line to the entire company. We created dashboards to continue to monitor these metrics. In future work, we would like to track metrics that other researchers and companies have found useful.

7 DISCUSSION AND RELATED WORK

While there is a large body of software engineering research work on code completions [11, 14, 17, 18, 22], there has been limited

deployment of these in large industrial environments [2–5]. With the advent of Generative AI, advances in the code assistance area are now in weeks rather than months. Several of these results are circulated as blog posts and not traditional papers, with limited discussion of evaluating the trade-offs between single-line and multi-line suggestions.

Hence, while multi-line support is common for AI code assistants, to the best of our knowledge, we are the first to describe contributions in:

(1) Using scope-based multi-line algorithm for AI-assisted suggestions
(2) Applying general LLM optimizations to improve the responsiveness of AI-assisted suggestions in an enterprise product
(3) Deploying A/B industrial-scale rollouts to quantify the effectiveness of multi-line vs single-line suggestions

7.1 Scope-Based Multi-Line Algorithm

While many tools provide multi-line suggestions, we did not find any discussion of the algorithm that they use to avoid the "jarring" effect or to thoughtfully limit the amount of code shown. For example, Amazon's CodeWhisperer [5] can be configured to produce multi-line suggestions, but the suggestions appear to run unlimited beyond the current semantic scope, causing a constant distraction for the user. They also seem to be preferentially triggered in highly specific contexts like a docstring, rather than through an automatic semantic-based algorithm as described in Section 3.

Google's ML code completion[4] does automatically constrain their multi-line suggestions based on a semantic filter, but this results in a far more restrictive algorithm compared to our approach. They show both the next token and the full line suggestion in a dropdown menu, with the length of the completion dependent on the current token. While the authors show a multi-line completion, it is unclear how many additional lines or what other restrictions are placed upon the completions, and no multi-line algorithm is described.

In contract, CodeCompose's multi-line algorithm automatically triggers as the user types – while remaining judicious in which trigger points are used, and selectively limiting the generated suggestion to the user's current scope. Multi-line suggestions are more difficult to accurately generate, but the scope-based algorithm enables us to display suggestions that fit within the user's current thinking context, thus supporting their train of thought without creating additional noise.

7.2 Responsive UX

With regards to latency reductions to make the product more responsive, many of the techniques we describe are standard in software engineering (e.g. early cancellation / streaming), or published techniques for optimizing transformer performance (e.g. Flash Attention[12], xFormers[8]). However there is little published research on the impact of deploying these in an industrial setting. While various external product releases, such as Copilot, Codeium, CodeWhisperer [6, 7], mention latency reductions as important drivers for improving the quality of the user experience, they do not offer a rigorous evaluation of latency impact across a suite of product metrics.

Hence, to the best of our knowledge, we are the first to contribute both an enumeration of the specific algorithms that drive these latency reductions, and a measurement of the impact of these when released in a production environment.

7.3 Evaluating Effectiveness of Multi-Line vs Single-Line Suggestions

As described in Section 5, it is not straightforward to determine whether multi-line suggestions are a net improvement to the product experience, as a series of rapid single-line suggestions could be received better by users. To the best of our knowledge, we are the first to show industry metrics from A/B rollouts that demonstrate the trade-off between single-line and multi-line suggestions. As Section 5.1 showed, our experiment metrics provide evidence for the hypothesis that despite latency tradeoffs, longer multi-line suggestions are more impactful to users than a series of shorter single-line suggestions.

Quantitative Comparisons. Although a direct comparison of quantitative results is impossible between products (given the different contexts, users, and tooling), when similar measures are reported, we can compare the overall trends. At Google, engineers [4] deployed a hybrid semantic ML code completion to 10k+ Google employees (over three months across eight programming languages). They observed a 6% reduction in coding iteration time (time between builds and tests) for users exposed to single-line ML completion compared to a control group. The authors measured a relatively low keystrokes savings: 0.6% saved from multi-line, compared to 7% of multi-line keystroke savings in CodeCompose. This differences indicates either lower display rate or lower number of lines in each suggestion generated by Google. The authors mention that a semantic filtering is applied during post-processing phase – this type of filtering is not applied in CodeCompose, and may explain a lower display rate in Google's product.

In contrast, at Meta, rather than using a semantic filter and dropdown menu constraint, CodeCompose completions are shown directly in the file, and are dependent on the position of the cursor. Hence, our suggestions automatically trigger in more cursor locations, likely leading to a higher reported display rate and overall throughput.

Amazon's CodeWhisperer [5] is a fully functional code completion tool integrated into the IDE. Amazon's analysis found that language models can generate code with correct syntax and pass unit tests in programming languages they are not intentionally trained on. The tool can be configured to produce multi-line suggestions, but these suggestions appear to trigger in more limited contexts and with significant latency for the user, making it unclear how effective multi-line is. We were unable to find any details from Amazon on the quantitative metric impact that single-line vs multi-line suggestions have on the overall product.

There have been several empirical evaluations of GitHub's Copilot [2] in actual use for automatic code completion. Nguyen et al. [16] used 33 LeetCode questions to create queries for Copilot in four different programming languages. They found that Copilot's Java suggestions have the highest correctness score (57%) while

JavaScript is the lowest (27%). Overall, Copilot's suggestions had low complexity with no notable differences between the programming languages. They also observed some potential Copilot shortcomings: such as generating code that can be further simplified and code that relies on undefined helper methods [16].

Qualitative Comparisons. Not all industry usage of code completions has been positive. A user study [21] with 24 participants to understand how programmers use and perceive Copilot found that while it did not necessarily improve the task completion time or success rate, most participants preferred to use it in daily programming tasks, since Copilot often provided a useful starting point and saved the effort of searching online. However, participants did face difficulties in understanding, editing, and debugging code snippets generated by Copilot, which significantly hindered their task-solving effectiveness [21].

These mixed results were also echoed by Bird *et al.* [10], that found while developer perceived an acceleration in coding, they often had to spend much more time reviewing the generated code. On the other hand, some other studies found that generated suggestions helped in discovering new APIs [9].

Our prior work on CodeCompose found an overwhelming 91.5% positive feedback for single-line completions accelerating the coding and allowing developers to discover new APIs across the monorepo [15]. Likewise, the user feedback described in Section 5.2 shows that users found a noticeable improvement in their experience as multi-line suggestions rolled out, indicating success with generating longer blocks of boilerplate code and more complex suggestions that help with discovery. Thus our work lays a foundation for the broader community to continue investing in this direction with generating multi-line suggestions while maintaining a positive user experience.

8 CONCLUSION AND CONTRIBUTION

In this paper, we make the following contributions related to the challenges we addressed:

- **Challenge 1. The Jarring Effect**: We developed the following scope-based algorithm (1) Multi-line suggestions are triggered only when the cursor is at the end of scope (2) Suggestions are shown until the end of the current block (3) After the suggestion is accepted, the cursor needs to be moved to the end of the suggested block.
- **Challenge 2. Responsive UX**: Multi-line suggestions take significantly longer to generate; hence we had to reduce the perceived user latency to ensure user adoption compared to single-line suggestions. This was done by (i) adding a UI indicator so users were aware of that a multi-line suggestion as being generated (ii) running optimizations to the model hosting service (e.g. Flash Attention, persistent K-V cache)
- **Challenge 3. Production Release Effectiveness**: During the roll-out of multi-line suggestions, we monitored metrics like acceptance rate, display rate, latency, and throughput to evaluate the net benefit of multi-line suggestions vs. single-line suggestions. These metrics demonstrated that the investment in multi-line suggestions disproportionately increased throughput, as multi-line suggestions accounted for 42% of total characters accepted per user per day (despite only accounting for at 16% of displayed suggestions), and drove the significant net increase in percentage of keystrokes saved nearly doubling from 9% to 17%. Less than 1% of engineers opted out and disabled multi-line suggestions.

9 ACKNOWLEDGEMENTS

We want to acknowledge and thank the following people for their work, help, and support in building the multi-line CodeCompose experience at Meta: Dian Belanger, Michael Bolin, Renuka Fernandez, Negar Ghorbani, Kelly Hirano, Diana Hsu, Kristian Kristensen, Killian Murphy, Chris Nixon, Zach Rait, Marshall Roch, Shahin Sefati, and Yiru Zhu.

REFERENCES

[1] Meta Accessed 2021. *ELI5: Bento - Interactive Notebook that Empowers Development Collaboration and Best Practices*. Meta. https://developers.facebook.com/blog/post/2021/09/20/eli5-bento-interactive-notebook-empowers-development-collaboration-best-practices/

[2] GitHub Accessed 2021. *GitHub Copilot*. GitHub. https://github.com/features/copilot

[3] Microsoft Accessed 2021. *Microsoft Intellicode*. Microsoft. https://visualstudio.microsoft.com/services/intellicode

[4] Google Accessed 2021. *ML Enhanced Code Completion*. Google. https://ai.googleblog.com/2022/07/ml-enhanced-code-completion-improves.html

[5] Amazon Accessed 2023. *Amazon CodeWhisperer*. Amazon. https://aws.amazon.com/codewhisperer

[6] Accessed 2023. Codeium vs Amazon CodeWhisperer. https://codeium.com/compare/comparison-codewhisperer-codeium

[7] Accessed 2023. Smarter, more efficient coding: GitHub Copilot goes beyond Codex with improved AI model. https://github.blog/2023-07-28-smarter-more-efficient-coding-github-copilot-goes-beyond-codex-with-improved-ai-model/

[8] GitHub Accessed 2023. *xformers*. GitHub. https://github.com/facebookresearch/xformers/tree/main/xformers

[9] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. 7, OOPSLA1, Article 78 (apr 2023), 27 pages. https://doi.org/10.1145/3586030

[10] Christian Bird, Denae Ford, Thomas Zimmermann, Nicole Forsgren, Eirini Kalliamvakou, Travis Lowdermilk, and Idan Gazit. 2023. Taking Flight with Copilot: Early insights and opportunities of AI-powered pair-programming tools. *Queue* 20, 6 (jan 2023), 35–57. https://doi.org/10.1145/3582083

[11] Marcel Bruch, Martin Monperrus, and Mira Mezini. 2009. Learning from examples to improve code completion systems. In *Proceedings of the 7th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on The foundations of software engineering (ESEC/FSE '09)*.

[12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. arXiv:2205.14135 [cs.LG]

[13] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen tau Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A Generative Model for Code Infilling and Synthesis. arXiv:2204.05999 [cs.SE]

[14] Seohyun Kim, Jinman Zhao, Yuchi Tian, and Satish Chandra. 2021. Code Prediction by Feeding Trees to Transformers. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. 150–162. https://doi.org/10.1109/ICSE43902.2021.00026

[15] Vijayaraghavan Murali, Chandra Maddila, Imad Ahmad, Michael Bolin, Daniel Cheng, Negar Ghorbani, Renuka Fernandez, Nachiappan Nagappan, and Peter C Rigby. 2024. AI-assisted Code Authoring at Scale: Fine-tuning, deploying, and mixed methods evaluations. In *Proceedings of the Foundations of Software Engineering (FSE '24)*.

[16] Nhan Nguyen and Sarah Nadi. 2022. An empirical evaluation of GitHub copilot's code suggestions. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR '22)*.

[17] Sebastian Proksch, Johannes Lerch, and Mira Mezini. 2015. Intelligent Code Completion with Bayesian Networks. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 25, 1, Article 3 (12 2015).

[18] R. Robles and M. Lanza. 2008. How Program History Can Improve Code Completion. In *2008 23rd IEEE/ACM International Conference on Automated Software Engineering*. 317–326. https://doi.org/10.1109/ASE.2008.42

[19] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom

Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. arXiv:2308.12950 [cs.CL]

[20] Qianhua Shan, David Sukhdeo, Qianying Huang, Seth Rogers, Lawrence Chen, Elise Paradis, Peter C. Rigby, and Nachiappan Nagappan. 2022. Using nudges to accelerate code reviews at scale *(ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 472–482. https://doi.org/10.1145/3540250.3549104

[21] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems (CHI EA '22)*.

[22] Wen Zhou, Seohyun Kim, Vijayaraghavan Murali, and Gareth Ari Aye. 2022. Improving Code Autocompletion with Transfer Learning. In *2022 IEEE/ACM 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 161–162. https://doi.org/10.1145/3510457.3513061